

Hypergraph Placement Model for R/A/B/C Hamiltonian Application

Uni20 Developer Documentation

June 6, 2026

1 Scope

This note defines the scheduling domain and cost functional for applying a block-sparse effective Hamiltonian in the R/A/B/C form used by the current TensorContraction bridge. The immediate target is the two-site DMRG Lanczos operator

$$R_r += f_t A_a B_b C_c, \quad t = (r, a, b, c, f_t), \quad (1)$$

where B is the input Krylov vector block family and R is the output Krylov vector block family.

For Hamiltonian application inside Lanczos, B and R must have the same logical block space and the same persistent device layout. The next Lanczos matvec consumes the previous output as its input, so an implicit relayout after each application is not acceptable. The same R/A/B/C machinery can also be used for environment construction, but then the input and output families are not necessarily the same object and need not share a device layout. This document records that distinction, but focuses first on the Hamiltonian case.

2 The Sparse Coefficient Tensor as a Hypergraph

Let

- $\mathcal{D}$ be the set of CUDA devices visible to one MPI rank,
- $\mathcal{K}$ be the logical center-block set shared by B and R ,
- $\mathcal{A}$ and $\mathcal{C}$ be the left and right environment block sets,
- $\mathcal{T}$ be the set of nonzero R/A/B/C terms.

Each term

$$t \in \mathcal{T}, \quad t = (r(t), a(t), b(t), c(t), f_t) \quad (2)$$

is a typed hyperedge connecting one output block, one input center block, and one left/right environment pair. Symmetry explains much of the sparsity, but the optimizer should use the actual nonzero tensor f as the ground-truth connectivity object.

3 Hamiltonian Scheduling Domain

The first useful Hamiltonian domain consists of four classes of choices: center-block ownership, active environment placement, exact factorization of the sparse coefficient tensor into temporary families, and placement of the first-stage product contributions and second-stage operations.

Center-block owner. Choose one canonical owner for each center block:

$$h : \mathcal{K} \rightarrow \mathcal{D}. \quad (3)$$

For Hamiltonian application, $h(k)$ is both the persistent owner of input block B_k and the final owner of output block R_k .

Active environment placement. Choose where active environment blocks are resident during the local solve:

$$e_A : \mathcal{A} \rightarrow \mathcal{D}, \quad (4)$$

$$e_C : \mathcal{C} \rightarrow \mathcal{D}. \quad (5)$$

The first model treats each active environment block as having one selected device. A later model may allow environment replicas, but then the second-stage device must choose one of the available replicas.

Exact path factorization. Split the sparse coefficient tensor into a left-first part and a right-first part:

$$f_{rabc} = f_{rabc}^L + f_{rabc}^R. \quad (6)$$

Each part is then represented by an exact sparse factorization through temporary labels:

$$f_{rabc}^L = \sum_{p \in \mathcal{P}_L} \lambda_{pab}^L \mu_{rpc}^L, \quad (7)$$

$$f_{rabc}^R = \sum_{q \in \mathcal{P}_R} \lambda_{qbc}^R \mu_{raq}^R. \quad (8)$$

This induces left-first temporaries

$$P_p^L = \sum_{a,b} \lambda_{pab}^L A_a B_b \quad (9)$$

and right-first temporaries

$$P_q^R = \sum_{b,c} \lambda_{qbc}^R B_b C_c. \quad (10)$$

The final output is

$$R_r += \sum_{p,c} \mu_{rpc}^L P_p^L C_c + \sum_{a,q} \mu_{raq}^R A_a P_q^R. \quad (11)$$

All products accumulated into one temporary must have the same dense matrix shape, and all downstream uses must be shape-compatible with the target output block. The factorization is therefore typed by both block labels and matrix dimensions; it is not merely an algebraic factorization of scalar coefficients.

This formulation incorporates the scalar prefactor before temporary accumulation. For example, if two first-stage products are reused with opposite signs downstream, the exact factorization may create

$$P_1^L = A_1 B_1 + A_2 B_2, \quad P_2^L = A_1 B_1 - A_2 B_2, \quad (12)$$

so that different C blocks can multiply different linear combinations. This is a symbolic sparse factorization, not a numerical low-rank approximation. The first optimizer should not use SVD, QR, or any other floating-point rank reduction of f to create the P objects: even if such a reduction lowers the formal rank, it changes numerical stability and reproducibility properties.

The term-level model is a degenerate exact factorization. A right-first term (r, a, b, c, f_t) can be represented with one temporary $q = (b, c)$ and coefficient $\mu_{raq}^R = f_t$, or with a private temporary $q = t$. The useful factorization groups terms only when doing so creates an exactly reusable linear combination.

Conservative no-rank-reduction specialization. Because we do not currently know a guaranteed stable useful factorization of f , the first implementation should use a conservative exact specialization. For left-first terms, use

$$p = (r, c), \quad \lambda_{(r,c)ab}^L = f_{abc}^L, \quad \mu_{r'pc'}^L = \begin{cases} 1, & p = (r', c'), \\ 0, & \text{otherwise.} \end{cases} \quad (13)$$

Thus

$$P_{rc}^L = \sum_{a,b} f_{abc}^L A_a B_b, \quad R_r += P_{rc}^L C_c. \quad (14)$$

For right-first terms, use

$$q = (r, a), \quad \lambda_{(r,a)bc}^R = f_{abc}^R, \quad \mu_{r'a'q}^R = \begin{cases} 1, & q = (r', a'), \\ 0, & \text{otherwise.} \end{cases} \quad (15)$$

Thus

$$P_{ra}^R = \sum_{b,c} f_{abc}^R B_b C_c, \quad R_r += A_a P_{ra}^R. \quad (16)$$

This specialization preserves the general λ/μ notation while avoiding any numerical factorization. It also fixes the natural second-stage destination: P_{rc}^L must be assembled where C_c resides, and P_{ra}^R must be assembled where A_a resides.

Product and second-stage placement. Choose the device for each nonzero first-stage product contribution:

$$\kappa_L(p, a, b) \in \mathcal{D} \quad \text{when } \lambda_{pab}^L \neq 0, \quad (17)$$

$$\kappa_R(q, b, c) \in \mathcal{D} \quad \text{when } \lambda_{qbc}^R \neq 0. \quad (18)$$

These choices create device-labelled partial temporaries. Also choose the device for each nonzero second-stage use:

$$\sigma_L(r, p, c) \in \mathcal{D} \quad \text{when } \mu_{rpc}^L \neq 0, \quad (19)$$

$$\sigma_R(r, a, q) \in \mathcal{D} \quad \text{when } \mu_{raq}^R \neq 0. \quad (20)$$

For the conservative specialization above, the first natural restriction is

$$\sigma_L(r, (r, c), c) = e_C(c), \quad (21)$$

$$\sigma_R(r, a, (r, a)) = e_A(a). \quad (22)$$

If a product contribution is accumulated on a different device from a second-stage consumer of its temporary, that partial temporary must be migrated and accumulated on the consumer device. Thus temporary migration is not an optional extension: it is part of the induced execution DAG.

The tuple $(h, e_A, e_C, \mathcal{P}_L, \mathcal{P}_R, \lambda, \mu, \kappa, \sigma)$ is still not the fully general CUDA scheduling space. It excludes arbitrary temporary reduction trees and nontrivial final-output reduction trees. These exclusions keep the first optimizer tractable while capturing the main placement tradeoffs.

4 Induced Execution DAG

Given the placement and factorization tuple above, the execution DAG is determined by the following canonical rules.

Input-center availability. For each center block b , define the devices that need B_b :

$$U_B(b) = \{\kappa_L(p, a, b) : \lambda_{pab}^L \neq 0 \text{ for some } p, a\} \cup \{\kappa_R(q, b, c) : \lambda_{qbc}^R \neq 0 \text{ for some } q, c\}. \quad (23)$$

The block is already resident on $h(b)$. If $d \in U_B(b)$ and $d \neq h(b)$, the model includes a peer copy or staging operation for B_b to device d . There is no reason to replicate B_b onto a device that has no assigned term using it.

Environment availability. Similarly, define

$$U_A(a) = \{\kappa_L(p, a, b) : \lambda_{pab}^L \neq 0 \text{ for some } p, b\} \cup \{\sigma_R(r, a, q) : \mu_{raq}^R \neq 0 \text{ for some } r, q\}, \quad (24)$$

$$U_C(c) = \{\kappa_R(q, b, c) : \lambda_{qbc}^R \neq 0 \text{ for some } q, b\} \cup \{\sigma_L(r, p, c) : \mu_{rpc}^L \neq 0 \text{ for some } r, p\}. \quad (25)$$

For steady-state resident Lanczos modelling, active A and C blocks may be assumed already materialized on the devices that need them. For cold-start or environment-staging models, the cost functional should include the required A and C staging bytes.

First-stage temporaries. Each device constructs local partial temporaries from the first-stage products assigned to that device:

$$P_p^{L,d} = \sum_{\substack{a,b \\ \lambda_{pab}^L \neq 0 \\ \kappa_L(p,a,b)=d}} \lambda_{pab}^L A_a B_b, \quad (26)$$

$$P_q^{R,d} = \sum_{\substack{b,c \\ \lambda_{qbc}^R \neq 0 \\ \kappa_R(q,b,c)=d}} \lambda_{qbc}^R B_b C_c. \quad (27)$$

The coefficients λ are applied during this accumulation, so identical matrix products that appear with different signs or scalars can feed distinct temporary objects.

Temporary migration. For each left-first temporary, define its consumer devices

$$V_L(p) = \{\sigma_L(r, p, c) : \mu_{rpc}^L \neq 0\}. \quad (28)$$

For each right-first temporary, define

$$V_R(q) = \{\sigma_R(r, a, q) : \mu_{raq}^R \neq 0\}. \quad (29)$$

If a nonzero partial $P_p^{L,d}$ exists and $v \in V_L(p)$ with $v \neq d$, then $P_p^{L,d}$ is copied to v and accumulated into the assembled temporary on v . The analogous rule applies to $P_q^{R,d}$. A later scheduler may choose tree or multicast strategies, but the first model counts one materialized partial-temporary transfer per source and consumer device.

Second-stage accumulation. Each nonzero μ_{rpc}^L computes and accumulates $\mu_{rpc}^L P_p^{L, \sigma_L(r,p,c)} C_c$ on $\sigma_L(r, p, c)$. Each nonzero μ_{raq}^R computes and accumulates $\mu_{raq}^R A_a P_q^{R, \sigma_R(r,a,q)}$ on $\sigma_R(r, a, q)$. The result is accumulated into a per-device partial output block

$$P_r^d. \quad (30)$$

Final output reduction. For each output block r , all nonzero partials P_r^d are reduced or copied to the canonical owner $h(r)$. If $d = h(r)$, the partial can be accumulated directly into the final R_r buffer. If $d \neq h(r)$, the model includes a peer transfer and accumulation into R_r on $h(r)$.

5 Cost Functional

The objective is predicted makespan. For each device d , define a device cost

$$C_d = C_d^{\text{gemm}} + C_d^{\text{copy}} + C_d^{\text{overhead}} + C_d^{\text{memory}}. \quad (31)$$

The optimizer minimizes

$$\min \max_{d \in \mathcal{D}} C_d, \quad (32)$$

possibly subject to memory-capacity constraints.

5.1 GEMM Work

Let $\Phi_{ab}^{L,1}$ be the flop count for $A_a B_b$, and $\Phi_{bc}^{R,1}$ be the flop count for $B_b C_c$. Let $\Phi_{pc}^{L,2}$ be the flop count for $P_p^L C_c$, and let $\Phi_{aq}^{R,2}$ be the flop count for $A_a P_q^R$. Then

$$\begin{aligned} C_d^{\text{gemm}} = & \alpha_{L,1} \sum_{\substack{p,a,b \\ \lambda_{pab}^L \neq 0 \\ \kappa_L(p,a,b)=d}} \Phi_{ab}^{L,1} + \alpha_{R,1} \sum_{\substack{q,b,c \\ \lambda_{qbc}^R \neq 0 \\ \kappa_R(q,b,c)=d}} \Phi_{bc}^{R,1} \\ & + \alpha_{L,2} \sum_{\substack{r,p,c \\ \mu_{rpc}^L \neq 0 \\ \sigma_L(r,p,c)=d}} \Phi_{pc}^{L,2} + \alpha_{R,2} \sum_{\substack{r,a,q \\ \mu_{raq}^R \neq 0 \\ \sigma_R(r,a,q)=d}} \Phi_{aq}^{R,2}. \end{aligned} \quad (33)$$

The coefficients α are not just reciprocal peak FLOP rates. For small block GEMMs they should absorb cuBLAS dispatch overhead and the reduced efficiency of small matrices.

Temporary construction also includes accumulation operations. For large dense blocks these are lower order than GEMMs, but for many tiny symmetry blocks they can contribute measurable kernel-launch and memory-bandwidth overhead. The overhead model below should therefore count the number of product accumulations into each temporary, not only the number of final temporary objects.

5.2 Data Movement

Let $\text{bytes}(M_i)$ be the storage size of block M_i . The steady-state input-center movement cost is

$$C_d^{B-\text{copy}} = \beta_B \sum_{\substack{b \in \mathcal{K} \\ d \in U_B(b) \\ d \neq h(b)}} \text{bytes}(B_b). \quad (34)$$

Cold-start environment staging can be modelled as

$$C_d^{AC-\text{stage}} = \beta_A \sum_{\substack{a \in \mathcal{A} \\ d \in U_A(a)}} \text{bytes}(A_a) + \beta_C \sum_{\substack{c \in \mathcal{C} \\ d \in U_C(c)}} \text{bytes}(C_c). \quad (35)$$

This term should normally be omitted or amortized when modelling repeated resident Lanczos matvecs with fixed active environments. If active environments are already resident according to e_A and e_C , then only additional replicas should be charged:

$$C_d^{AC-\text{replica}} = \beta_A \sum_{\substack{a \in \mathcal{A} \\ d \in U_A(a) \\ d \neq e_A(a)}} \text{bytes}(A_a) + \beta_C \sum_{\substack{c \in \mathcal{C} \\ d \in U_C(c) \\ d \neq e_C(c)}} \text{bytes}(C_c). \quad (36)$$

Which of these two terms is appropriate depends on whether the model is scoring cold environment materialization or steady-state resident matvecs.

The final output movement cost is

$$C_d^{R-\text{reduce}} = \rho_R \sum_{\substack{r \in \mathcal{K} \\ P_r^d \neq 0 \\ d \neq h(r)}} \text{bytes}(R_r). \quad (37)$$

In the restricted domain, this is a flat peer-copy-plus-accumulate model. A future model may replace it with an explicit reduction tree.

Temporary migration is

$$S_L(p) = \{\kappa_L(p, a, b) : \lambda_{pab}^L \neq 0\}, \quad (38)$$

$$S_R(q) = \{\kappa_R(q, b, c) : \lambda_{qbc}^R \neq 0\}. \quad (39)$$

The corresponding movement cost is

$$C_d^{P-\text{copy}} = \beta_P \left(\sum_{\substack{p \in \mathcal{P}_L \\ s \in S_L(p) \\ d \in V_L(p) \\ d \neq s}} \text{bytes}(P_p^L) + \sum_{\substack{q \in \mathcal{P}_R \\ s \in S_R(q) \\ d \in V_R(q) \\ d \neq s}} \text{bytes}(P_q^R) \right). \quad (40)$$

This term is essential. A schedule that saves first-stage GEMMs but ships a large temporary to many devices may be worse than recomputing a smaller temporary locally.

5.3 CUDA and Scheduling Overheads

CUDA overheads are not represented well by FLOPs or bytes alone. The cost model should include count features such as

$$C_d^{\text{overhead}} = \gamma_T N_d^{\text{term}} + \gamma_L N_d^{L-\text{prod}} + \gamma_R N_d^{R-\text{prod}} + \gamma_{PC} N_d^{P-\text{copy}} + \gamma_{PR} N_d^{\text{partial}} + \gamma_Q N_d^{\text{peer}}, \quad (41)$$

where $N_d^{L-\text{prod}}$ and $N_d^{R-\text{prod}}$ count first-stage product accumulations on d , $N_d^{P-\text{copy}}$ counts temporary migrations, N_d^{partial} counts nonzero partial output blocks on d , and N_d^{peer} counts peer-staged input blocks. These coefficients should be fitted empirically from no-trace fixture benchmarks. They are the main way the model captures kernel launch overhead, cuBLAS handle/stream effects, small-GEMM overhead, peer-copy setup overhead, and duplicate first-stage group costs.

5.4 Temporary Memory Pressure

Let M_d be the peak temporary memory required by first-stage temporaries, temporary replicas, peer input replicas, and partial outputs on device d under the chosen schedule. The first optimizer can enforce a hard constraint

$$M_d \leq M_d^{\max} \quad (42)$$

or add a soft penalty

$$C_d^{\text{memory}} = \mu M_d. \quad (43)$$

The hard constraint is preferable for production scheduling. The soft penalty is useful while fitting a prototype model.

6 Integer-Program View

For a fixed exact temporary factorization, the restricted placement domain can be linearized. Introduce binary variables

$$o_{k,d} = 1 \quad \text{if center block } k \text{ is owned by device } d, \quad (44)$$

$$a_{i,d} = 1 \quad \text{if environment block } A_i \text{ is resident on } d, \quad (45)$$

$$c_{i,d} = 1 \quad \text{if environment block } C_i \text{ is resident on } d, \quad (46)$$

$$y_{pab,d}^L = 1 \quad \text{if product } A_a B_b \text{ contributes to } P_p^L \text{ on } d, \quad (47)$$

$$y_{qbc,d}^R = 1 \quad \text{if product } B_b C_c \text{ contributes to } P_q^R \text{ on } d, \quad (48)$$

$$s_{rpc,d}^L = 1 \quad \text{if } P_p^L C_c \text{ for } R_r \text{ runs on } d, \quad (49)$$

$$s_{raq,d}^R = 1 \quad \text{if } A_a P_q^R \text{ for } R_r \text{ runs on } d. \quad (50)$$

The assignment constraints are

$$\sum_{d \in \mathcal{D}} o_{k,d} = 1 \quad \text{for every } k \in \mathcal{K}, \quad (51)$$

$$\sum_{d \in \mathcal{D}} a_{i,d} = 1 \quad \text{for every } i \in \mathcal{A}, \quad (52)$$

$$\sum_{d \in \mathcal{D}} c_{i,d} = 1 \quad \text{for every } i \in \mathcal{C}, \quad (53)$$

$$\sum_{d \in \mathcal{D}} y_{pab,d}^L = 1 \quad \text{for every nonzero } \lambda_{pab}^L, \quad (54)$$

$$\sum_{d \in \mathcal{D}} y_{qbc,d}^R = 1 \quad \text{for every nonzero } \lambda_{qbc}^R, \quad (55)$$

$$\sum_{d \in \mathcal{D}} s_{rpc,d}^L = 1 \quad \text{for every nonzero } \mu_{rpc}^L, \quad (56)$$

$$\sum_{d \in \mathcal{D}} s_{raq,d}^R = 1 \quad \text{for every nonzero } \mu_{raq}^R. \quad (57)$$

Activation variables for B replicas, A/C staging, temporary migrations, partial outputs, and peer reductions can be constrained above the corresponding y and s variables. The conservative specialization can add constraints $s_{r,(r,c),c,d}^L = c_{c,d}$ and $s_{r,a,(r,a),d}^R = a_{a,d}$ so the final temporary accumulation happens where the corresponding environment block is resident. The makespan objective is then

$$\min T \quad \text{subject to} \quad C_d \leq T \quad \text{for every } d \in \mathcal{D}. \quad (58)$$

This form is a mixed-integer linear program once the cost coefficients are fixed. It can be used offline as an oracle for small fixtures. The runtime optimizer can use cheaper local search or graph partitioning heuristics against the same evaluator.

7 Relation to Current Implementation

The current deterministic resident executor is a strict subcase:

$$\mathcal{P}_L = \emptyset, \tag{59}$$

$$q = (b, c, d), \tag{60}$$

$$P_q^R = B_b C_c \quad \text{constructed separately for each device } d \text{ that needs it,} \tag{61}$$

$$\kappa_R(q, b, c) = d, \tag{62}$$

$$\sigma_R(r, a, q) = h(r) = d. \tag{63}$$

That is, every term is right-first, each right-first temporary is local to the owner of the output block that needs it, identical (B_b, C_c) temporaries are reused only within one device, and cross-device temporary migration is not modelled. The current **empirical-contiguous** policy is a legacy two-device ordered-range diagnostic in the existing block construction order. That ordering is not a mathematical invariant: block spaces can be permuted and any chosen partition can be packed contiguously afterward. Therefore an ordered-range policy is not an appropriate optimization domain beyond diagnostic comparison with the real hypergraph placement model.

The next model should separate logical block identity, placement, and storage packing:

1. choose an exact temporary factorization and placement variables over logical block and term keys;
2. evaluate the induced copies, first-stage groups, partial outputs, and reductions;
3. pack the chosen placement into coalesced device slabs as a storage implementation detail.

8 Environment Construction Variant

For environment construction, the same term hypergraph and path labels apply, but the Hamiltonian-specific constraint

$$\text{owner}(B_k) = \text{owner}(R_k) \tag{64}$$

is not required. The more general domain uses separate owner maps

$$h^B : \mathcal{B} \rightarrow \mathcal{D}, \quad h^R : \mathcal{R} \rightarrow \mathcal{D}. \tag{65}$$

The output may also be written to host-side cached environment storage after construction rather than retained as the next Krylov input. This changes the output-reduction and staging terms, but not the underlying typed hypergraph.